

Design and Development of a Multi-Platform Peer-to-Peer Marketplace and Standalone Escrow Engine Supporting Fiat and TRON (TRX) Blockchain Exchanges

A unified transaction-trust layer for goods, services, and digital assets across formal marketplaces and informal social-media channels

Prepared For	Rev. Dr. Kwame Ofosuhen Peasah, Department of Computer Science
Prepared By	Group 2 ASENSO, Owusu Ansah BAAH, Fredrick ASARE, Foster Solomon Owusu
Date	June 8, 2026
Project Type	Third-Year Mini Project, Department of Computer Science
Duration (Proposed)	Eight (8) Weeks

1. Executive Summary

Online peer-to-peer commerce has rapidly expanded beyond structured marketplaces. A large and growing share of trade in goods, digital assets, and informal services now takes place directly through social-media networks and messaging applications such as WhatsApp, Telegram, and dedicated cryptocurrency communities. These informal channels operate completely outside traditional marketplace safeguards, exposing participants to fraud, non-delivery, payment disputes, and identity misrepresentation.

This proposal sets out the design and development of a **dual-purpose platform** delivering an integrated peer-to-peer marketplace for goods listings alongside a **standalone escrow engine** that can secure independently negotiated external transactions. The marketplace will operate as the first internal consumer of the engine, while external users will be able to generate shareable escrow links and QR codes — bringing trusted intermediation to deals that originate over chat and social media.

The platform will be delivered across Web and Mobile clients with multi-asset settlement: simulated fiat/mobile-money for physical goods, milestone-based release for digital services, and real cryptocurrency escrow on the **TRON (TRX) public testnet** accessed via the **TronGrid API**. The intended outcome is a focused, demonstrable trust layer that addresses a clear and currently unmet need in informal digital commerce.

2. Problem Statement

Existing online marketplaces provide transaction support and dispute resolution strictly within their own walled-garden platforms. However, a significant and growing volume of peer-to-peer commerce now takes place outside these platforms, leaving four core problems unsolved:

- **No Trust Layer for Informal Channels.** Transactions negotiated over WhatsApp, Telegram, and cryptocurrency communities completely lack any trusted intermediary mechanism, leaving both parties exposed to fraud, non-delivery, and payment disputes.
- **Fragmented Settlement Methods.** Real-world P2P deals span fiat / mobile money, digital deliverables, and cryptocurrencies, yet no accessible platform offers a unified escrow workflow across all three.
- **Identity Misrepresentation Risk.** Informal trading channels provide no consistent identity verification, making impersonation and fake-listing fraud trivially easy.
- **Research Gap.** Existing academic work on P2P systems focuses heavily on network architecture, resource sharing, and internal reputation models, rather than on practical, deployable escrow infrastructure for informal external commerce.

3. Proposed System Scope

The proposed system is organised around nine functional pillars. The central design principle is that the *Escrow Engine* is an independent infrastructure layer; the marketplace is simply its first internal consumer, and external users access the same engine through generated transaction links.

3.1 Integrated P2P Marketplace

A native marketplace module supporting product listings for phones, laptops, electronics, and digital products. Users will be able to browse categories, list items, message counterparties, and complete purchases. When a buyer commits to a purchase, the marketplace will automatically create an escrow transaction by invoking the core engine, rather than handling funds itself.

3.2 Standalone Escrow Link & QR Generator

A minimal external portal where two parties who have already negotiated terms elsewhere (for example over WhatsApp or Telegram) can quickly create a custodial escrow. One party specifies the amount, asset type, and release conditions; the system issues a shareable link and QR code; the counterparty joins, funds are locked, and the engine handles the rest of the lifecycle.

3.3 Central Escrow Engine (State Machine)

The heart of the platform. The engine implements a strict state machine that transitions every transaction through the following phases: **Created** → **Funded** → **Delivered / Under Review** → **Disbursed or Disputed**. The same engine serves marketplace purchases and standalone external escrows; both routes converge into one auditable lifecycle.

3.4 Multi-Asset Settlement Adapters

Modular settlement components plug into the engine to handle different asset types:

- **Fiat / Mobile Money adapter** — simulated mobile-money flow for physical goods and informal services.

- **Digital Goods adapter** — milestone-based release markers for digital deliverables such as code, designs, or files.
- **TRON (TRX) blockchain adapter** — real on-chain escrow on the TRON public testnet (Shasta or Nile), accessed via the TronGrid API.

3.5 User Verification & Authentication

Email and phone-based account verification, optional lightweight KYC fields, and session management for both web and mobile clients. The verification layer is required for marketplace activity and is also available to external escrow users who wish to raise their counterparty trust signals.

3.6 Dispute Resolution Module

When either party flags a problem, the engine freezes the escrow and exposes a structured dispute workflow. Both parties submit evidence; an administrator (or designated arbiter) reviews and triggers the appropriate state transition. Time-locked auto-resolution rules are applied to escrows that remain inactive past a configured deadline.

3.7 Cross-Platform Mobile Application

A Flutter (or React Native) mobile client providing marketplace browsing, transaction history, escrow status tracking, and on-the-go escrow link / QR generation. The mobile client and the web client consume the same backend API, so behaviour is consistent across platforms.

3.8 TRON Blockchain Integration

Cryptocurrency escrow is delivered on the TRON public testnet because of its fast block finality and negligible gas fees, both of which are advantageous for an academic project. Rather than operating a dedicated TRON node, the backend will use the **TronGrid API** to query the network, observe TRX deposits to assigned escrow wallets, and verify state changes before releasing funds.

3.9 Admin Monitoring Dashboard

A web-only dashboard giving the project team operational visibility: active escrows by state, pending disputes, settlement adapter health, and TronGrid integration status. The dashboard exists for demonstration, evaluation, and supervisor review.

4. Proposed Technical Architecture

4.1 Layered Architecture

The system is split into three tightly integrated layers, with settlement adapters branching off the core. The escrow engine sits at the centre — independent of the marketplace — so external users can access it directly.

Presentation Layer	Web Client (React.js / Vue.js) · Mobile App (Flutter / React Native)
Business Logic Layer	Marketplace Module (listings, browse, cart) · Standalone Module (link & QR generator)
Core Infrastructure	Central Escrow Engine — state locking, timeouts, dispute routing, audit log

Settlement Adapters	Fiat / Mobile Money gateway (simulated) · Digital goods release marker · TRON testnet via TronGrid API
----------------------------	--

4.2 Technology Stack

Layer	Proposed Technology	Purpose
Web Frontend	React.js (Vue.js as alternative), Tailwind CSS	Marketplace storefront, admin dashboard, escrow web portal
Mobile Frontend	Flutter (React Native as alternative)	Single codebase serving Android and iOS clients
Backend	Node.js (Express) — Java Spring Boot as alternative	REST APIs, escrow state machine, settlement orchestration
Database	PostgreSQL (MySQL as alternative)	Listings, user profiles, escrow lifecycle records, audit log
Authentication	JWT-based sessions with email/phone verification	User registration, session management, optional KYC fields
Blockchain Gateway	TronGrid API	Querying the TRON network, observing TRX deposits, broadcasting release transactions, verifying state changes
Hosting	Render / AWS Free Tier for backend; Vercel or Netlify for web	Continuous deployment of demo environment

4.3 Escrow Transaction Lifecycle

Every transaction — whether originating inside the marketplace or via an external link — passes through the same deterministic lifecycle managed by the engine:

Created	→	Both parties have joined; terms are recorded.
Funded	→	Buyer's funds are locked by the engine or held on the TRON testnet for crypto escrows.
Delivered / Under Review	→	Seller has fulfilled; buyer is confirming or evaluating delivery.
Disbursed	→	Buyer approves; engine releases funds to the seller.
Disputed	→	Either party raises a problem; engine freezes the escrow and opens evidence review.

5. Methodology

Given the eight-week window, the project will follow a tight iterative methodology with weekly milestones. Work is parallelised across the three team members with sharply defined ownership boundaries, and a uniform backend API ensures that the web and mobile clients consume the same code paths for every escrow operation. Source control is maintained on Git/GitHub with small,

frequent commits and pull-request reviews.

- **Requirements gathering:** targeted surveys of fellow students to map common challenges, fraud types, and trust points faced during informal transactions on messaging apps.
- **Literature review:** analysis of existing P2P trading platforms, digital escrow frameworks, and decentralised trust models.
- **Design:** system architecture, database schema, REST API contract, and UI mockups for both the marketplace and the escrow-link flow.
- **Implementation:** backend-first vertical slices — escrow engine state transitions are built first and validated, then the web and mobile clients are wired to the same endpoints.
- **Evaluation:** functional testing of every module, simulated dispute resolution scenarios end-to-end, TRON testnet transaction verification, and a small usability evaluation with student volunteers.
- **Deployment:** continuous deployment of the demo environment (Render / Vercel) so the supervisor can inspect progress at any point.

6. Proposed Timeline (8 Weeks)

Tasks are distributed so that the three team members work in parallel as soon as the shared API contract is fixed at the end of Week 2. The role assignments below indicate primary responsibility; pair-programming and review will occur across boundaries.

Week	Focus	Deliverable	Primary Responsibility
Week 1	Requirements gathering & literature review	Targeted student surveys, review of existing P2P platforms and escrow frameworks, documented requirements specification.	All members
Week 2	System design & API contract	Finalised system architecture, database schema, REST API contract, UI wireframes for web and mobile; repository, CI and hosting environments configured.	All members (ASARE leads design)
Week 3	Backend foundations & authentication	Database implementation, user registration / authentication endpoints, account and listing schemas; first API endpoints live for the other members to consume.	BAAH, Fredrick
Week 4	Central escrow engine & TronGrid integration	Full implementation of the escrow lifecycle (Created → Funded → Delivered → Disbursed / Disputed), with audit logging, configurable time-locks, and TronGrid listeners for TRX deposit detection on the TRON testnet.	BAAH, Fredrick
Week 5	Web client & admin dashboard	Web marketplace storefront, listing creation, browse and search, admin monitoring dashboard, web-based escrow link / QR generator.	ASENSO, Owusu Ansah

Week	Focus	Deliverable	Primary Responsibility
Week 6	Cross-platform mobile application	Mobile marketplace browsing, transaction history, escrow status tracking, and on-the-go escrow link / QR generation built on the shared backend API.	ASARE, Foster Solomon Owusu
Week 7	Settlement adapters, dispute module & integration	Fiat / mobile-money simulated adapter, digital-goods release marker, TRON testnet settlement adapter completion, dispute resolution workflow, end-to-end web-mobile-backend integration.	All members (BAAH leads adapters)
Week 8	Testing, evaluation & project report	Functional tests, simulated dispute flows, TRON testnet transaction verification, usability evaluation with student volunteers, bug fixes, written project report and demonstration prepared for submission.	All members

7. Risks & Mitigation

Risk	Mitigation
Custodial liability concerns around holding real funds	Fiat and mobile-money flows are simulated end-to-end; only TRX on the public testnet is moved on-chain, so no real money is ever held.
TronGrid API rate limits or testnet downtime	Local mock adapter that mimics TRX deposit confirmations; engine logic is identical regardless of which adapter is active, allowing offline demos.
Multi-platform scope (Web + Mobile + Backend + Blockchain) is broad	Backend-first vertical slices; uniform REST API; web and mobile share the same code paths. Pure UI polish is the first thing dropped if pressure rises.
Dispute resolution complexity (timeouts, partial deliveries, edge cases)	Strict state-machine design with documented allowed transitions and configurable time-locks; complex edge cases captured as test scenarios rather than UI features.
User-trust concerns around a small project team holding escrow	Clear in-app messaging that the platform is an academic prototype; all real-money screens are watermarked 'Simulation'.
Eight-week window remains demanding for a three-person team	Weekly vertical-slice delivery; deferred-features list maintained from Week 1; non-essential polish (e.g. extra payment categories) trimmed before any core pillar.

8. Expected Impact

By extending escrow protection beyond walled-garden marketplaces and into the informal trading channels where most modern peer-to-peer commerce actually happens, the platform addresses a clear and currently unmet need. Treating the escrow engine as standalone infrastructure that any

consumer (the marketplace, an external user, or a future third-party application) can plug into provides a credible academic and architectural contribution beyond a generic marketplace prototype.

For students participating in informal trading communities, the platform offers a practical reduction in fraud and dispute risk. For the department, the project demonstrates a multi-platform, multi-asset system design that integrates web, mobile, and public-blockchain components within a single coherent backend.

9. Conclusion & Next Steps

The proposed marketplace and standalone escrow platform addresses a clear, growing student and community need with a focused technical solution. The architecture chosen — a uniform REST backend consumed by a React web client and a Flutter mobile client, with multi-asset settlement adapters culminating in TRON testnet integration via TronGrid — is well-suited to parallel, iterative delivery by a three-person team within the eight-week mini-project window.

Approval is respectfully requested to commence the project under the scope, methodology, and timeline set out in this proposal.